

Security Test Report

Test ID & Vector Name	Objective & Scope	Execution Code	Expected vs Actual Result	Analyze	Status
TC-SEC-001 BOLA /Data Leakage Broken Access Control	Testing whether a user who is not the property owner can leak owner's private offer data by directly manipulating the API address.	<pre>fetch('http://localhost:5000/api/property/7', { method: 'GET', headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer eyJhbGciOi...'` } }) .then(r => r.json()) .then(d => console.log(d));</pre>	Expected: Access denied (403 Forbidden) or active data masking / filtering. Actual: HTTP 200 OK {"success":true,"count":0,"data":[]}	The user's token was successfully authenticated; however, the ownership verification mechanism at the database layer detected that the requesting user is not included in the authorized owner list for that specific listing.	PASS
TC-SEC-002 SQL Injection	Testing if the search input sanitizes dangerous characters to prevent unauthorized database access.	Entering the following payload sequence directly into the Search Interface : ` OR 1=1 --`	Expected: Input is processed as plain text, safely returning zero results without causing any database errors. Actual: HTTP 200 OK {"success":true,"data":[]}	The database safely handled the raw input as a harmless plain text string. This confirms that the mechanism is fully functional, completely preventing SQL-based data leakage.	PASS
TC-SEC-003 Stored XSS	Testing if the property description field escapes harmful script tags to prevent XSS-based session and user data theft.	Body: <pre><script> alert('XSS: ' + localStorage.getItem('token')); </script></pre>	Expected: The system escapes harmful script tags and processes them as plain text. No code triggers on other users' screens. Actual: No alert box or popup triggered on owner's screen. The payload was rendered safely as a harmless literal description string. .	The application safely processed the incoming tags as harmless plain text instead of executable browser code. This confirms that framework-level automatic escaping is fully functional, completely preventing unauthorized session and user data theft.	PASS

TC-SEC-004 Mass Assignment Insecure Design	Testing if a user can modify hidden fields (isAdmin, role) during sign-up to steal unauthorized privileges..	<pre>fetch('/api/auth/register', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ username: 'hacker_yusuf', email: 'yusuf@gmail.com', password: 'Password123*', isAdmin: true, role: 'admin' }) }).then(r => r.json()).then(d => console.log(d));</pre>	Expected: The backend validation logic detects invalid data structures and drops any unauthorized parameters (like isAdmin or role) from the request. Actual: HTTP 400 Bad Request {error: 'Validation failed', details: [...]}	The server blocked unauthorized parameters in the incoming request from being written directly to the database.	PASS
TC-SEC-005 NoSQL Query Injection	Testing whether MongoDB logical operators (">": "") injected into input fields can penetrate the system and bypass the password validation mechanism.	<pre>fetch('/api/auth/login', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ email: 'elif@gmail.com', password: { ">": "" } }) }).then(r => r.json()).then(d => console.log(d));</pre>	Expected: The system detects unexpected data types or objects in the input and processes them strictly as plain text or rejects the request safely. Actual: HTTP 400 Bad Request {error: 'Validation failed', details: [...]}	The server successfully blocked the malicious data payload before it could reach the database layer. This confirms that strict data type checking (Strict Schema Validation) is enforced on public-facing APIs, completely preventing cyber attacks from penetrating the database.	PASS
TC-SEC-006 Rate Limiting Security Misconfiguration	Testing if the system detects and limits rapid automated login attempts to stop credential cracking. j	<pre>for(let i=1;i<=50;i++){ fetch('/api/auth/login',{ method:'POST', headers:{'Content-Type':'application/json'}, body:JSON.stringify({ email:'elif@gmail.com', password:i===50?'RealPass1*':'Wrong_'+i }) }).then(r=>console.log(r.status)); }</pre>	Expected: HTTP 429 Too Many Requests or account lockout after a few failed attempts. Actual: HTTP 200 OK { "success":true,"token":"eyJhbG..." }	"The endpoint processed all automated login requests without hitting any rate limits. The lack of a Rate Limiter middleware leaves user accounts highly exposed to brute-force attacks."	FAIL

DevTools is now available in Turkish

Don't show again Always match Chrome's language Switch DevTools to Turkish

Back to listings

No images available for this property

home
istanbul

Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a gidin.

4 / 47 requests | 0.8 kB / 6.7 kB tra | Line 1, Column 37

Reliabay

'OR 1=1 --

No matches found
We couldn't find anything for "'OR 1=1 --".

DevTools is now available in Turkish

Don't show again Always match Chrome's language Switch DevTools to Turkish

search?q=%27%20OR%201=1...

1 {"success":true,"data":[]}

1 / 2 requests | 0.3 kB / 0.3 kB tran | 26 characters selected

villa
istanbul

\$2.500

About this property

<script>alert('XSS_Vulnerability_Detected: ' + localStorage.getItem('token'));</script>

Make an Offer

Save for Later

Kullanıcı adı

Email

Şifre

Şifreyi doğrula

City

isBuvur?

DevTools is now available in Turkish

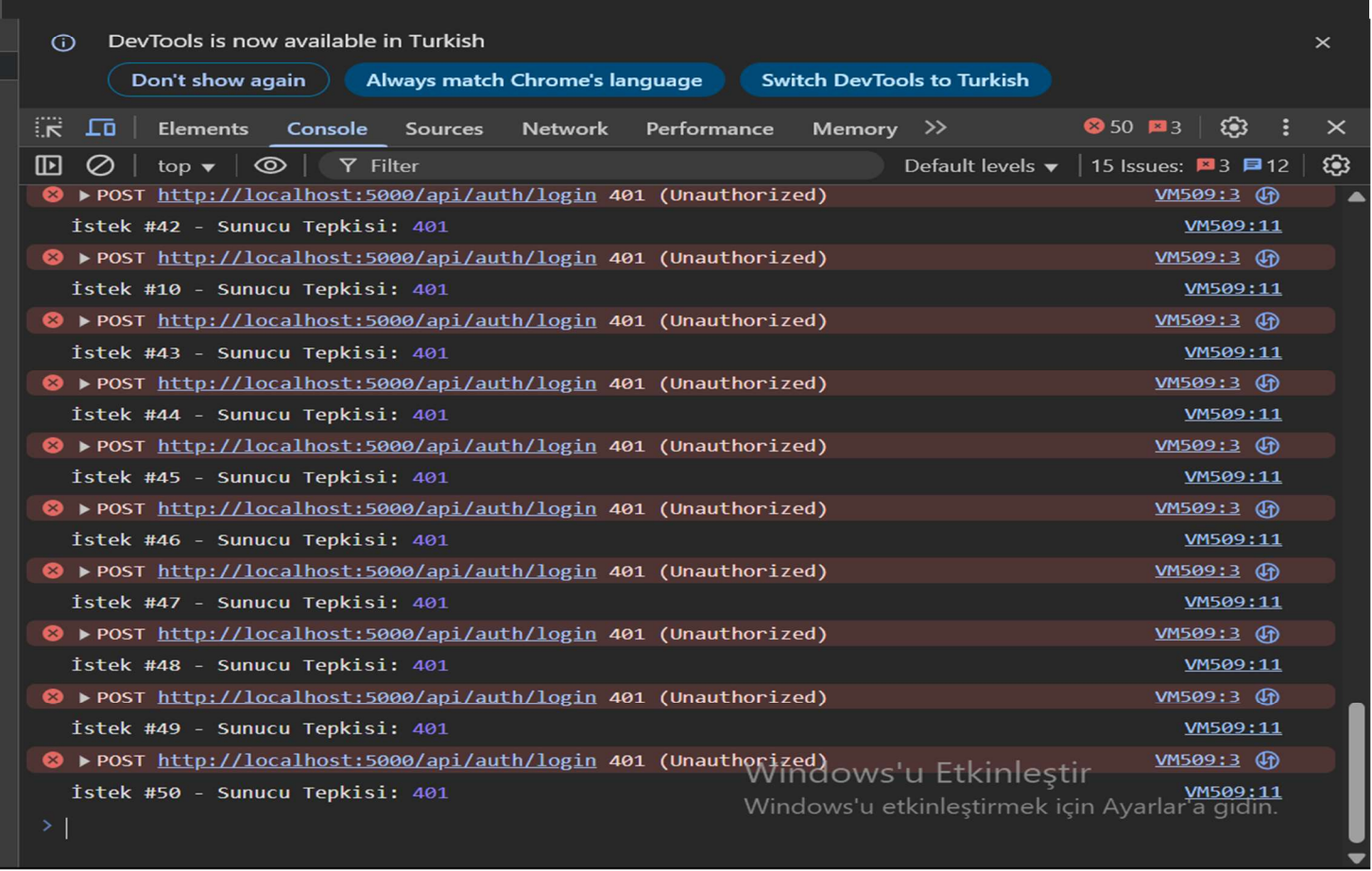
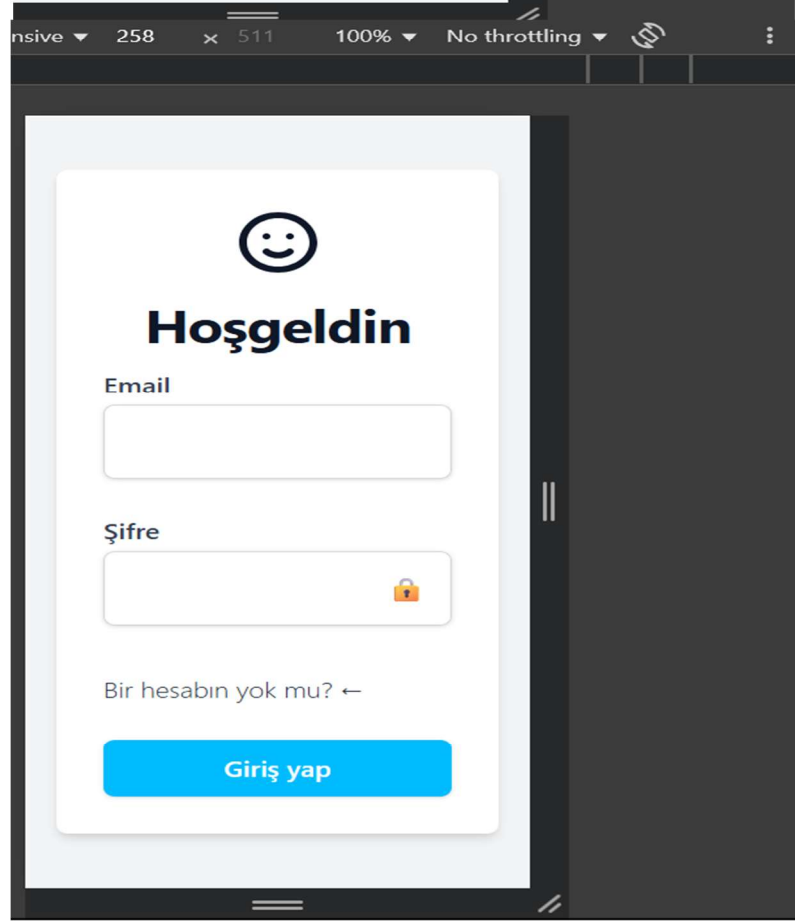
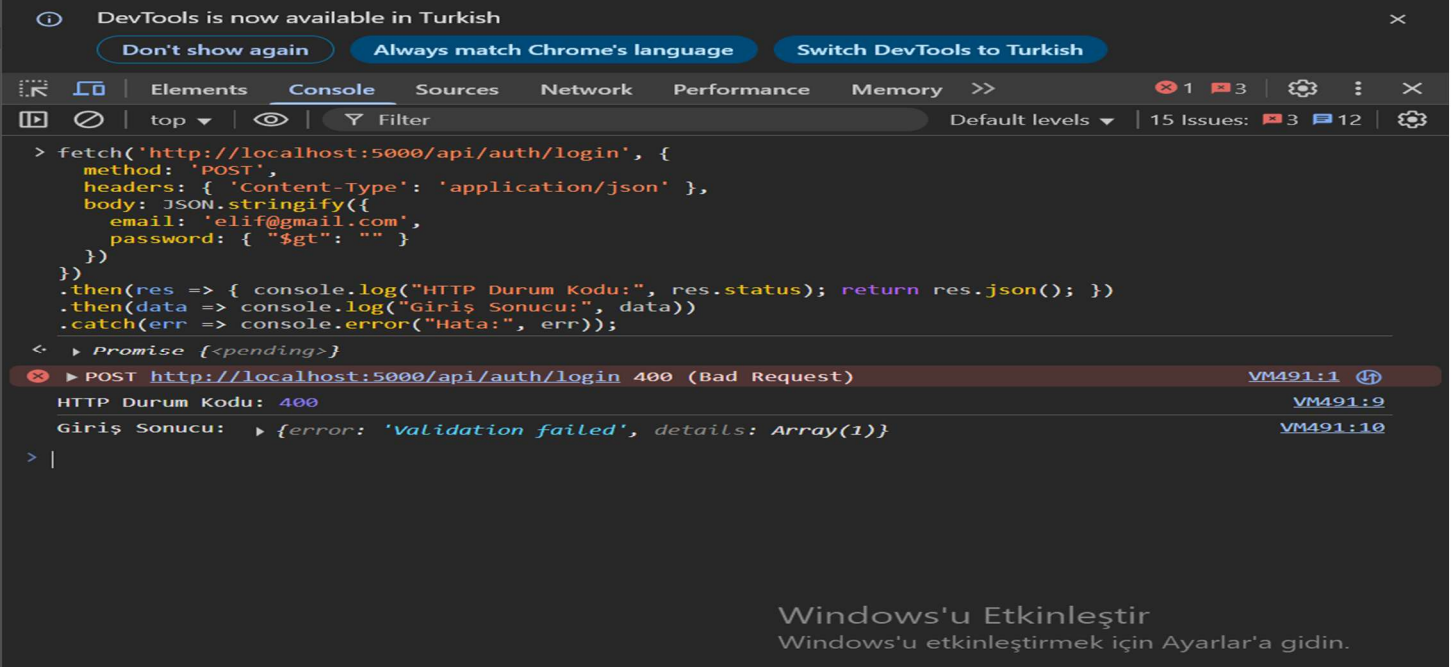
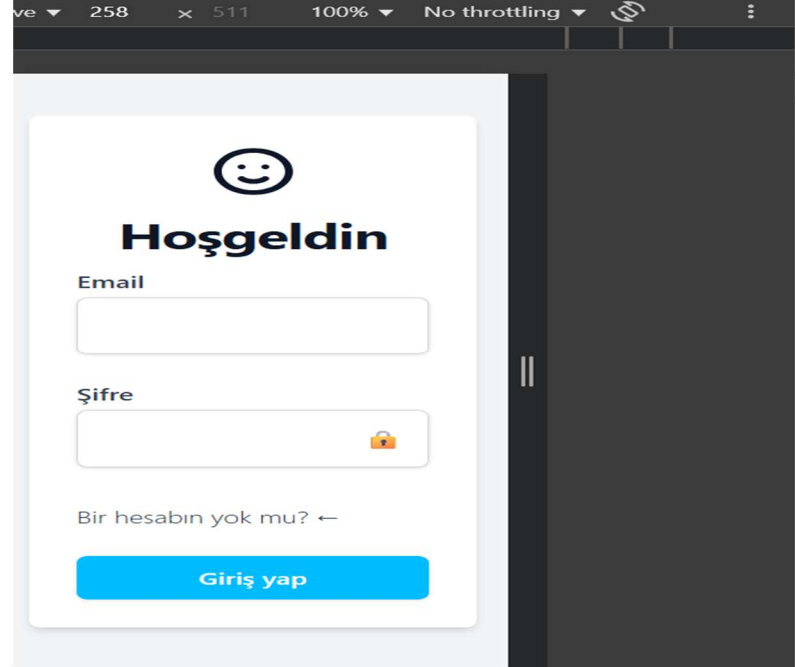
Don't show again Always match Chrome's language Switch DevTools to Turkish

```
fetch('http://localhost:5000/api/auth/register', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: 'hacker_yusuf',
    email: 'yusuf_hacker@gmail.com',
    password: 'Password123',
    // Saldırı parametreleri (Arayüzde olmayan gizli roller)
    isAdmin: true,
    role: 'admin'
  })
})
.then(res => res.json())
.then(data => console.log("Gelen Yanıt:", data));
```

POST http://localhost:5000/api/auth/register 400 (Bad Request) VM465:1

Gelen Yanıt: {error: 'Validation failed', details: Array(3)}

Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a gidin.



Security Testing Process with OWASP ZAP

To find the security problems of our web application, we used the OWASP ZAP tool. The process consists of two main steps:

- 1. **AJAX Spidering:** First, we ran the AJAX Spider. This tool automatically clicks all buttons, links, and forms to explore the complete structure and all hidden pages of our modern web application.
- 2. **Active Scan:** After mapping the website, we started an Active Scan. ZAP sent test attacks to our server to find real security risks.

As a result of these two steps, we obtained the security report and compiled the risk table shown.

Finding Name	Category	Risk Level	Description
Content Security Policy (CSP) Not Set	Insecure Design	Medium	No CSP header defined. Allows browsers to load dynamic scripts from unauthorized external sources, heavily increasing XSS risks.
Cookie No HttpOnly / Without Secure Flag	Cryptographic / Auth Failures	Medium	Cookies lack HttpOnly and Secure flags. This allows hackers to steal login tokens
Missing Anti-clickjacking Header	Security Misconfiguration	Medium	Missing X-Frame-Options allows clickjacking.
Strict-Transport-Security Not Set	Cryptographic Failures	Medium	Without HSTS, the website can be forced to use insecure HTTP instead of HTTPS.

Finding Name	Category	Risk Level	Description
Cross-Domain Misconfiguration	Security Misconfiguration	Medium	The server allows untrusted websites to read its data due to weak CORS settings. This leads to unauthorized data sharing.
CSP: Failure to Define Directive with No Fallback	Insecure Design	Medium	The CSP header is misconfigured and does not block untrusted scripts. This allows hackers to execute custom XSS attacks
Server / X-Powered-By Version Leakage	Information Disclosure	Low	HTTP headers show the exact server software and version numbers (like Express or Tomcat). This makes it easy for hackers to find known exploits
X-Content-Type-Options Header Missing	Security Misconfiguration	Low	X-Content-Type-Options is missing. Browsers may try to execute non-executable files (like images).
Cookie SameSite Attribute Missing	Broken Access Control	Low	Without the SameSite flag, cookies can be sent automatically during cross-site requests, exposing user sessions to CSRF attacks.
Loosely Scoped Cookie	Broken Access Control	Low	The cookie domain is too broad. This allows untrusted subdomains to access sensitive data.
Private IP Disclosure	Information Disclosure	Info	Internal IP addresses are leaked in error messages. This helps attackers map the private network structure.

Finding Name	Category	Risk Level	Description
Sensitive Information in URL / Comments	Information Disclosure	Info	Development notes (like TODO tags) are visible in the source code. This leaks system details and future code updates to attackers.
Content-Type Header Missing	Security Misconfiguration	Info	Without the Content-Type header, the browser cannot determine the correct file format.

